

Esercizio degli errori nel codice-Assistenza Virtuale

Python-Revisione Tecnica

Studente: Simone Santonico

Data Analisi: 17/04/2026

```
1  import datetime
2
3  while True
4      comando_utente = input("Cosa vuoi sapere? ")
5      if comando_utente == "esci":
6          print("Arrivederci!")
7          break
8      else:
9          print(assistente_virtuale(comando_utente))
10
11 def assistente_virtuale(comando):
12     if comando == "Qual è la data di oggi?":
13         oggi = datetime.datetime.today()
14         risposta = "La data di oggi è " + oggi.strftime("%d/%m/%Y")
15     elif comando == "Che ore sono?":
16         ora_attuale = datetime.datetime.now().time()
17         risposta = "L'ora attuale è " + ora_attuale.strftime("%H:%M")
18     elif comando == "Come ti chiami?":
19         risposta = "Mi chiamo Assistente Virtuale"
20     else:
21         risposta = "Non ho capito la tua domanda."
22     return risposta
```

Obiettivo dell'esercizio - Analizzare il codice sopra indicato, senza eseguirlo, individuando:

- Il comportamento del programma;
- Le casistiche non gestite e non standard;
- Gli errori di sintassi e/o logici presenti;
- Le possibili soluzioni per ciascun problema.

Questa analisi esamina il funzionamento di un semplice assistente virtuale scritto in Python, evidenziando una serie di criticità che non compromettono affidabilità e chiarezza.

Sono stati individuati **sette errori principali**.

- **Analisi degli Errori**

La funzione viene dichiarata dopo essere stata utilizzata

Codice coinvolto:

```
while True: ... print(assistente_virtuale(comando_utente)) # riga 9 def
assistente_virtuale(comando): # riga 11 ...
```

Perché rappresenta un problema: Il ciclo while viene eseguito immediatamente e, se l'utente non digita "esci", il programma tenta di richiamare `assistente_virtuale()` prima che questa sia stata definita. Il risultato è un errore del tipo:

NameError: name 'assistente_virtuale' is not defined

Come sarebbe più corretto: Le funzioni dovrebbero essere dichiarate prima del codice che le utilizza. Una buona pratica consiste nel posizionare tutte le definizioni nella parte superiore del file o racchiudere il codice principale in:

```
if __name__ == "__main__":
```

Errore #2 — Severità: Media

Indentazione errata del comando break

Codice coinvolto:

```
if comando_utente == 'esci': # riga 5 print('Arrivederci!') # riga 6 break # riga 7
```

Perché rappresenta un problema:

Il break non è all'interno del blocco if, ma allo stesso livello del ciclo while. Questo significa che il ciclo termina sempre dopo la prima iterazione, indipendentemente dal comando inserito

Come sarebbe più corretto:

Il break deve essere indentato allo stesso livello del print. In Python l'indentazione determina la struttura logica del programma, non è un dettaglio estetico.

Errore #3 — Severità: Media

Confronto delle stringhe troppo rigido

Codice coinvolto:

```
if comando == 'Qual è la data di oggi?' elif comando == 'Che ore sono?' elif  
comando == 'Come ti chiami?'
```

Perché rappresenta un problema:

Il confronto è sensibile a maiuscole, minuscole e punteggiatura. Basta una minima variazione per far fallire il riconoscimento del comando.

Come sarebbe più corretto:

Prima del confronto, normalizzare l'input:

```
comando = comando.strip().lower()
```

Oppure utilizzare parole chiave:

```
if "data" in comando:
```

Errore #4 — Severità: Alta Uso poco sicuro della concatenazione con +

Codice coinvolto:

```
"La data di oggi è " + oggi.strftime('%d/%m/%Y') "L'ora attuale è " +  
ora_attuale.strftime('%H:%M')
```

Perché rappresenta un problema:

Anche se strftime() restituisce una stringa, concatenare con + è fragile: se uno dei valori non fosse una stringa, verrebbe generato un TypeError.

Come sarebbe più corretto:

Utilizzare le f-string, più leggibili e sicure:

```
f"La data di oggi è {oggi:%d/%m/%Y}"
```

Errore #5 — Severità: Alta

Mancanza dei due punti dopo While True

Codice coinvolto:

```
while True # riga 3
```

Perché rappresenta un problema:

Le strutture di controllo in Python devono terminare con `:`. L'assenza genera immediatamente un errore di sintassi:

SyntaxError: expected ':'

Come sarebbe più corretto:

while True: con i due punti.

Errore #6 — Severità: Alta**Variabile risposta non inizializzata in anticipo****Codice coinvolto:**

```
def assistente_virtuale(comando): if ...: risposta = ... elif ...: risposta = ... else:  
risposta = ... return risposta
```

Perché rappresenta un problema:

La variabile viene creata solo all'interno dei rami condizionali. Se il ramo `else` venisse modificato o rimosso, il `return` causerebbe un errore:

UnboundLocalError: local variable 'risposta' referenced before assignment

Come sarebbe più corretto:

Inizializzare la variabile all'inizio della funzione:

risposta = "Non ho capito la tua domanda."

Errore #7 — Severità: Alta

Metodo errato: `datetime.datetime.today()`

Codice coinvolto:

oggi = datetime.datetime.today()

Perché rappresenta un problema:

Il metodo non esiste. La forma corretta è:

```
datetime.date.today()
```

Altrimenti Python genera:

```
AttributeError: module 'datetime' has no attribute 'datetoday'
```

Come sarebbe più corretto:

```
oggi = datetime.date.today()
```

Conclusioni e Raccomandazioni

L'analisi ha messo in luce **sette errori significativi**, cinque dei quali di gravità elevata. Il programma, così com'è, non può funzionare correttamente e presenta diversi punti deboli che ne riducono la robustezza.

consiglio:

- Definire sempre le funzioni prima del loro utilizzo.
- Controllare attentamente la sintassi delle strutture di controllo.
- Curare l'indentazione, che in Python determina la logica del programma.
- Normalizzare l'input dell'utente prima di confrontarlo.
- Preferire le f-string alla concatenazione con +.
- Inizializzare sempre le variabili prima dell'uso.
- Verificare i nomi dei metodi consultando la documentazione ufficiale.